

Table of Contents

Object-Oriented Programming — Concepts

Course Outline | 16 Weeks | 1 Hour/Week

Course Description

Course Schedule Overview

Course Architecture

Week 1 — 27 May – 2 June

Introduction to Object-Oriented Programming

Objectives

Topics

Conceptual Diagram

Java Illustration

Procedural vs OOP — Side-by-Side

Test Checkpoint

Common Mistakes

Extension Challenges

Week 2 — 3 June – 9 June

Classes & Objects

Objectives

Topics

Memory Model (Conceptual)

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 3 — 10 June – 16 June

Attributes, Methods & Object State

Objectives

Topics

State Diagram (Conceptual)

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 4 — 17 June – 23 June

Encapsulation

Objectives

Core Concept

Real-World Analogy

Access Modifier Table

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 5 — 24 June – 30 June

Constructors & Object Initialization

Objectives

Core Concept

Java Illustration

Constructor Chaining Diagram

Test Checkpoint

Common Mistakes

Extension Challenges

Week 6 — 1 July – 7 July

Inheritance — Concept & IS-A Relationship

Objectives

Core Concept

Real-World Analogy

IS-A Test

Java Illustration

Inheritance Hierarchy (ASCII)

Test Checkpoint

Common Mistakes

Extension Challenges

Week 7 — 8 July – 14 July

Method Overriding & the `super` Reference

Objectives

Core Concept
Overriding vs Overloading
Java Illustration
Test Checkpoint
Common Mistakes
Extension Challenges
Week 8 — 15 July – 21 July

★ MID-TERM EXAM ★

Exam Coverage (Weeks 1–7)
Sample Exam Questions (Preview)
Exam Logistics
Week 9 — 22 July – 28 July

Polymorphism

Objectives
Core Concept
Two Forms of Polymorphism
Upcasting & Downcasting
Java Illustration
Test Checkpoint
Common Mistakes
Extension Challenges
Week 10 — 29 July – 4 August

Abstraction — Abstract Classes

Objectives
Core Concept
Real-World Analogy
Abstract Class Rules
Java Illustration
Test Checkpoint
Common Mistakes
Extension Challenges
Week 11 — 5 August – 11 August

Abstraction — Interfaces

Objectives

Core Concept

Abstract Class vs Interface

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 12 — 12 August – 18 August

Object Relationships: Association, Aggregation & Composition

Objectives

Core Concept

Relationship Types

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 13 — 19 August – 25 August

Exception Handling in OOP

Objectives

Core Concept

Exception Class Hierarchy

Java Illustration

Test Checkpoint

Common Mistakes

Extension Challenges

Week 14 — 26 August – 1 September

OOP Design Principles (SOLID Introduction)

Objectives

Core Concept

SOLID Overview

SRP Violation vs Correct Design

Test Checkpoint

Common Mistakes

Extension Challenges

Week 15 — 2 September – 8 September

Introduction to Design Patterns & Course Review

Objectives

Core Concept

Three Pattern Categories

Singleton Pattern

Builder Pattern

Course Review Checklist (All Weeks)

Week 16 — 9 September – 15 September

★ FINAL EXAM ★

Exam Coverage (Weeks 1–15)

Sample Final Questions (Preview)

Final Exam Logistics

Recommended Reading

Object-Oriented Programming — Concepts

Course Outline | 16 Weeks | 1 Hour/Week

Institution: IIC University of Technology
Course URL: <http://elearning.iic.edu.kh/course/view.php?id=1229>
Illustration Language: Java

Course Description

This course introduces the fundamental concepts and principles of **Object-Oriented Programming (OOP)**. Students will learn how to think in an object-oriented way — modeling real-world problems as interacting objects — and understand the four core pillars: **Encapsulation, Inheritance, Polymorphism, and Abstraction**. Java is used solely as a vehicle to illustrate and practice these concepts.

Course Schedule Overview

Week	Date Range	Topic	Type
1	27 May – 2 June	Introduction to OOP	Lecture
2	3 June – 9 June	Classes & Objects	Lecture + Lab
3	10 June – 16 June	Attributes, Methods & Object State	Lecture + Lab
4	17 June – 23 June	Encapsulation	Lecture + Lab
5	24 June – 30 June	Constructors & Object Initialization	Lecture + Lab
6	1 July – 7 July	Inheritance — Concept & IS-A Relationship	Lecture + Lab
7	8 July – 14 July	Method Overriding & the <code>super</code> Reference	Lecture + Lab
8	15 July – 21 July	MID-TERM EXAM	Exam

10	29 July – 4 August	Abstraction — Abstract Classes	Lecture + Lab
11	5 August – 11 August	Abstraction — Interfaces	Lecture + Lab
12	12 August – 18 August	Object Relationships (Association, Aggregation, Composition)	Lecture + Lab
13	19 August – 25 August	Exception Handling in OOP	Lecture + Lab
14	26 August – 1 September	OOP Design Principles (SOLID Intro)	Lecture
15	2 September – 8 September	Introduction to Design Patterns & Review	Review + Lab
16	9 September – 15 September	FINAL EXAM	Exam

Course Architecture

OOP Concepts – 16 Weeks

- PART 1: FOUNDATIONS (Weeks 1–3)
 - Week 1: What is OOP? History, Benefits, OOP vs Procedural
 - Week 2: Classes & Objects – The Core Building Blocks
 - Week 3: Attributes, Methods & Object State
- PART 2: THE FOUR PILLARS (Weeks 4–7)
 - Week 4: Encapsulation
 - Week 5: Constructors & Object Initialization
 - Week 6: Inheritance & IS-A Relationship
 - Week 7: Method Overriding & super
- ★ MID-TERM EXAM (Week 8) ★
- PART 3: ADVANCED PILLARS (Weeks 9–11)
 - Week 9: Polymorphism
 - Week 10: Abstraction – Abstract Classes
 - Week 11: Abstraction – Interfaces
- PART 4: OOP IN PRACTICE (Weeks 12–15)
 - Week 12: Object Relationships
 - Week 13: Exception Handling in OOP
 - Week 14: SOLID Design Principles

Week 1 — 27 May – 2 June

Introduction to Object-Oriented Programming

Objectives

By the end of this week, students will be able to:

- Explain what OOP is and why it was introduced
- Compare OOP with procedural programming
- Identify the four pillars of OOP
- Map real-world entities to OOP concepts

Topics

1. The history and motivation of OOP
2. Procedural vs Object-Oriented approach
3. Overview of the 4 OOP Pillars: Encapsulation, Inheritance, Polymorphism, Abstraction
4. Key terminology: class, object, attribute, method, instance
5. Real-world modeling with OOP

Conceptual Diagram

REAL WORLD	→	OOP CONCEPT
Car (blueprint/design)	→	Class
My red Toyota	→	Object (instance)
Color, speed, brand	→	Attributes (fields)
Drive, brake, honk	→	Methods (behaviors)

Java Illustration

```
// A Class is a blueprint – an Object is a real instance created from it
public class Car {
    String brand;
    String color;
    int speed;
```



```

        void accelerate() {            speed += 10;
        System.out.println(brand + " is now going " + speed + " km/h");    }
    void brake() {                    speed -= 10;
        System.out.println(brand + " slowed to " + speed + " km/h");    }    }
    public class Main {    public static void main(String[] args) {
        Car myCar = new Car();        // Object 1            myCar.brand = "Toyota";
        myCar.color = "Red";            myCar.speed = 0;            myCar.accelerate();
        Car friendCar = new Car();    // Object 2 – independent state
        friendCar.brand = "Honda";            friendCar.speed = 60;            friendCar.brake();
    }    }

```

Procedural vs OOP — Side-by-Side

PROCEDURAL	OBJECT-ORIENTED
Functions operate on data	Objects combine data + behavior
Global or passed-in state	Each object owns its own state
Hard to scale / reuse	Reusable via inheritance
No concept of "blueprint"	Classes as reusable blueprints

Test Checkpoint

- ☐ Can explain OOP in own words (no jargon)
- ☐ Can list and briefly define the 4 pillars
- ☐ Can identify the difference between a class and an object
- ☐ Can name 3 real-world examples and map them to classes/objects

Common Mistakes

Mistake	Clarification
Confusing class and object	Class = the blueprint; Object = the thing built from it
Thinking OOP is just about Java	OOP is a programming paradigm, applicable in many languages
Listing syntax, not concepts	Understanding <i>why</i> matters more than <i>how</i> at this stage

Extension Challenges

- **Basic:** List 5 real-world objects and identify their attributes and behaviors
- **Intermediate:** Compare an OOP approach vs a procedural approach for modeling a school system — write a paragraph for each

- **Advanced:** Sketch a class diagram (on paper) for a simple library system: `Book` , `Member` , `Librarian`

Week 2 — 3 June – 9 June

Classes & Objects

Objectives

- Define a class with fields and methods in Java
- Create and use multiple objects from one class
- Understand that each object has its own independent state
- Understand the `new` keyword and memory allocation concept

Topics

1. Anatomy of a class: fields, methods, access
2. Creating objects using `new`
3. Object references and memory (Heap vs Stack — conceptual)
4. `null` reference — what it means
5. Multiple objects from one class — independent state

Memory Model (Conceptual)

Stack (references)	Heap (actual objects)
s1	[Student Object] name = "Sokha" age = 20
s2	[Student Object] name = "Dara" age = 22
s3 = null	(points to nothing)

Java Illustration

```
public class Student {  
    // Fields - the state of each object  
    String name;
```

```

int    age;    String major;           // Methods – the behavior each object can perform
void introduce() {
    System.out.println("I am " + name + ", age " + age + ", studying " + major);    }
    String getSummary() {        return name + " (" + major + ")";    }    }
    public class Main {    public static void main(String[] args) {
Student s1 = new Student();        s1.name = "Sokha";        s1.age = 20;
s1.major = "Computer Science";        Student s2 = new Student();
s2.name = "Dara";        s2.age = 22;        s2.major = "Engineering";
s1.introduce();    // s1 and s2 are independent        s2.introduce();
System.out.println(s1.getSummary());    }    }

```

Test Checkpoint

- ☐ Class has at least 3 fields and 2 methods
- ☐ Two objects created independently with different states
- ☐ Method with a return value works correctly
- ☐ Can explain what happens in memory when `new Student()` is called

Common Mistakes

Mistake	Clarification
Class name does not match filename	<code>class Student</code> must be saved as <code>Student.java</code>
Forgetting <code>new</code>	<code>Student s = Student()</code> is invalid — must use <code>new</code>
Assuming objects share state	Each object has its own copy of fields

Extension Challenges

- **Basic:** Create a `Book` class with `title`, `author`, `pages` fields and a `printInfo()` method
- **Intermediate:** Create 3 `Book` objects, store them and print a formatted book list
- **Advanced:** Design a `Course` class that holds a reference to a `Teacher` object as one of its fields — demonstrate that fields can be objects themselves

Week 3 — 10 June – 16 June

Attributes, Methods & Object State

Objectives

- Distinguish between instance variables, local variables, and parameters
- Write methods that read and modify object state
- Use method parameters and return values effectively
- Understand the concept of object state changing over time

Topics

1. Instance variables (state) vs local variables (temporary)
2. Method parameters — passing data in
3. Return types — passing data out
4. Void vs non-void methods
5. Object state changes through method calls

State Diagram (Conceptual)

BankAccount Object

Initial State: balance = 1000.0

deposit(500) → balance = 1500.0

withdraw(200) → balance = 1300.0

withdraw(2000) → balance = 1300.0 (rejected – insufficient)

The object's state changes only through its methods.

Java Illustration

```
public class BankAccount {  
    // Instance variable – persists for the life of the object  
    double balance;  
    String owner;  
  
    void deposit(double amount) {  
        // 'amount' is a local parameter – only exists during this call  
        if (amount > 0) {  
            balance += amount;  
            System.out.println("Deposited: " + amount + " | Balance: " + balance);  
        }  
    }  
}
```

```

boolean withdraw(double amount) {
    if (amount > balance) {
        System.out.println("Insufficient funds.");
        return false;
    }
    balance -= amount;
    System.out.println("Withdrawn: " + amount + " | Balance: " + balance);
    return true;
}

double getBalance() {
    return balance; // return the current state
}

```

Test Checkpoint

- ☐ Instance variable is accessible across all methods in the class
- ☐ Local variable cannot be accessed outside its method
- ☐ Method with `boolean` return type returns `true` / `false` correctly
- ☐ Object state (balance) changes correctly after method calls

Common Mistakes

Mistake	Clarification
Using local variable instead of instance variable	Local variables reset every call — use instance variable for persistent state
Returning from a <code>void</code> method	<code>void</code> methods do not return a value
Not using the return value	<code>account.withdraw(200)</code> returns <code>boolean</code> — can be used in <code>if</code>

Extension Challenges

- **Basic:** Add a `printStatement()` method to `BankAccount` that prints current owner and balance
- **Intermediate:** Add a `transfer(BankAccount target, double amount)` method — withdraw from self, deposit to target
- **Advanced:** Design a `ShoppingCart` class with `addItem()`, `removeItem()`, `getTotalPrice()` — track item list and total

Week 4 — 17 June – 23 June

Encapsulation

Objectives

- Explain encapsulation as a core OOP principle
- Apply access modifiers (`private` , `public` , `protected`)
- Implement getters and setters with data validation
- Understand why hiding internal state is important

Core Concept

Encapsulation is the practice of bundling data (fields) and the methods that operate on that data into a single unit (class), and **restricting direct access** to the internal state of an object.

Real-World Analogy

ATM Machine

VISIBLE to user (public): `insertCard()`, `withdraw()`, `checkBalance()`

HIDDEN from user (private): vault mechanism, transaction logs, PIN validation

The user interacts through a controlled interface.

The internal logic is protected and cannot be tampered with.

Access Modifier Table

Modifier	Same Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
(default)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Java Illustration

```
public class Student {  
    // Private fields - hidden from outside  
    private String name;  
    private int age;  
    private double gpa;  
  
    // Public constructor  
    public Student(String name, int age, double gpa) {
```

```

    this.name = name;
    setAge(age);    // use setter for validation even inside constructor
    setGpa(gpa);    }    // Getters – read-only access
public String getName() { return name; }    public int    getAge() { return age; }
public double getGpa() { return gpa; }
// Setters – controlled, validated write access    public void setAge(int age) {
    if (age ≥ 16 && age ≤ 60) {    this.age = age;    } else {
        System.out.println("Invalid age. Defaulting to 18.");    this.age = 18;
    }    }    public void setGpa(double gpa) {
    if (gpa ≥ 0.0 && gpa ≤ 4.0) {    this.gpa = gpa;    } else {
        System.out.println("Invalid GPA. Defaulting to 0.0");    this.gpa = 0.0;
    }    }    @Override    public String toString() {
    return "Student{name='" + name + "', age=" + age + ", gpa=" + gpa + "}";    }
}

```

Test Checkpoint

- ☐ `private` field cannot be accessed directly from outside the class
- ☐ Setter correctly rejects invalid input (test with age = -5 or gpa = 10.0)
- ☐ Getter returns the correct field value
- ☐ `toString()` produces a readable output

Common Mistakes

Mistake	Clarification
Making getters/setters <code>private</code>	Defeats the purpose — they must be <code>public</code> to be useful
Setter with no validation	Encapsulation is meaningless if any value is accepted
Exposing internal collections directly	<code>return list</code> gives outside code direct access — return a copy or unmodifiable view

Extension Challenges

- **Basic:** Refactor the `BankAccount` from Week 3 — make `balance` private, add validated getter/setter
- **Intermediate:** Add a `transactionHistory` list to `BankAccount` — it should be `private` and only readable through a `getHistory()` method
- **Advanced:** Discuss in writing: what are the security and maintenance benefits of encapsulation in a banking application?

Constructors & Object Initialization

Objectives

- Explain the role of a constructor in object creation
- Write default and parameterized constructors
- Implement constructor overloading
- Use the `this` keyword for field disambiguation and constructor chaining

Core Concept

A **constructor** is a special method called automatically when an object is created. It sets up the object's initial state. A well-designed constructor ensures an object is **always created in a valid state**.

Java Illustration

```
public class Student {
    private String name;
    private int age;
    private String major;

    // Default constructor - sets safe defaults
    public Student() {
        this("Unknown", 18, "Undeclared"); // chains to the full constructor
    }

    // Partial constructor - name + major only
    public Student(String name, String major) {
        this(name, 18, major);
    }

    // Full parameterized constructor
    public Student(String name, int age, String major) {
        this.name = name; // 'this.name' = field; 'name' = parameter
        this.age = age;
        this.major = major;
    }

    @Override
    public String toString() {
```



```

        return name + " | Age: " + age + " | Major: " + major;    }    }
public class Main {    public static void main(String[] args) {
    Student s1 = new Student();                                // uses defaults
    Student s2 = new Student("Sokha", 20, "CS");                // fully specified
    Student s3 = new Student("Dara", "Engineering");            // partial
    System.out.println(s1);    System.out.println(s2);    System.out.println(s3);
}    }

```

Constructor Chaining Diagram

```

new Student()
├─ calls this("Unknown", 18, "Undeclared")
│   └─ calls Student(String, int, String)
│       └─ assigns: name, age, major

```

Test Checkpoint

- ☐ Default constructor produces valid object (no nulls or invalid values)
- ☐ Parameterized constructor assigns all fields correctly
- ☐ `this()` chain is the first statement in the constructor
- ☐ Constructor overloading provides flexible object creation

Common Mistakes

Mistake	Clarification
Adding return type to constructor	<code>void Student()</code> is a regular method, not a constructor
<code>this()</code> not on the first line	Java requires <code>this()</code> or <code>super()</code> to be the first statement
Confusing <code>this.name</code> and <code>name</code>	Without <code>this.</code> , the assignment <code>name = name</code> does nothing

Extension Challenges

- **Basic:** Add 2 constructors to `Course` class — one with name only, one with name + credits
- **Intermediate:** Create an `Enrollment` class whose constructor takes a `Student` object and a `Course` object
- **Advanced:** Explain why forcing object creation through constructors (rather than direct field assignment) leads to safer code — provide a concrete example

Inheritance — Concept & IS-A Relationship

Objectives

- Explain the concept and purpose of inheritance
- Identify IS-A relationships between classes
- Use `extends` to create a class hierarchy
- Use `super()` to call parent constructors
- Understand what is and is not inherited

Core Concept

Inheritance allows a class (child/subclass) to acquire the fields and methods of another class (parent/superclass). It promotes **code reuse** and establishes a natural hierarchy between related types.

Real-World Analogy

```
Person (general – has name, age, introduce())
├── Student (specific – also has major, gpa, study())
└── Teacher (specific – also has subject, teach())
```

IS-A Test

Question: Is a Student a Person? → YES → use inheritance
Question: Is a Car a Person? → NO → do not use inheritance
Question: Is a GradStudent a Student? → YES → valid multi-level inheritance

Rule: Only use inheritance when a true IS-A relationship exists.

Java Illustration

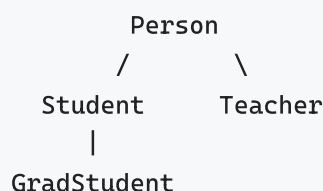
```
public class Person {
    protected String name;
    protected int age;
```

```

public Person(String name, int age) {      this.name = name;      this.age = age;
}
    public void introduce() {
        System.out.println("Hi, I am " + name + ", age " + age);    }    }
// Student IS-A Personpublic class Student extends Person {    private String major;
    public Student(String name, int age, String major) {
        super(name, age);    // call Person's constructor first        this.major = major;
    }
    public void study() {
        System.out.println(name + " is studying " + major);    }    }
// Teacher IS-A Personpublic class Teacher extends Person {    private String subject;
    public Teacher(String name, int age, String subject) {        super(name, age);
        this.subject = subject;    }        public void teach() {
        System.out.println(name + " teaches " + subject);    }    }

```

Inheritance Hierarchy (ASCII)



Test Checkpoint

- ☐ `super()` is the first statement in the child constructor
- ☐ Child class can call inherited methods (`introduce()`) without redefining them
- ☐ `instanceof Person` returns `true` for a `Student` object
- ☐ `private` fields from parent are NOT directly accessible in the child

Common Mistakes

Mistake	Clarification
Inheriting without an IS-A relationship	e.g., <code>Stack extends ArrayList</code> — Stack has-a list, it is not a list
Accessing parent's <code>private</code> fields	Use <code>protected</code> or getters in the parent
Forgetting <code>super()</code> when parent has no default constructor	Java will not auto-generate a call if the parent requires arguments

Extension Challenges

- **Basic:** Create `Animal` → `Dog` — Dog inherits `eat()`, adds `bark()`

- **Intermediate:** Build a 3-level hierarchy: `Person → Student → GradStudent`, each level adding meaningful fields and methods
- **Advanced:** Write a paragraph explaining why deep inheritance hierarchies (5+ levels) can be problematic in practice

Week 7 — 8 July – 14 July

Method Overriding & the `super` Reference

Objectives

- Explain method overriding as runtime behavior redefinition
- Apply `@Override` annotation correctly
- Use `super.method()` to extend parent behavior
- Understand when to override `toString()` and `equals()`
- Contrast overriding with overloading

Core Concept

Method Overriding occurs when a child class provides its own implementation of a method already defined in the parent class. The method signature must be identical. Java resolves which version to call at **runtime**, based on the actual object type.

Overriding vs Overloading

OVERRIDING	OVERLOADING
Defined in child class	Defined in same class
Same method name + same signature	Same method name + different params
Resolved at RUNTIME	Resolved at COMPILE TIME
Related to inheritance	Does not require inheritance
Example: <code>toString()</code> in subclass	Example: <code>print(int)</code> & <code>print(String)</code>

Java Illustration

```
public class Person {
    protected String name;
    protected int age;
}
```

```

public Person(String name, int age) {      this.name = name;      this.age = age;
}      public String getRole() {      return "Person";      }
@Override      public String toString() {
    return "[" + getRole() + "] " + name + " (age " + age + ")";    }
public class Student extends Person {      private String major;
    public Student(String name, int age, String major) {      super(name, age);
        this.major = major;    }      @Override      public String getRole() {
        return "Student";    // replaces "Person" at runtime    }      @Override
    public String toString() {    // Extend parent's toString - no duplication
        return super.toString() + " | Major: " + major;    }    }

```

Test Checkpoint

- ☐ `@Override` annotation causes a compile error if the method signature does not match any parent method
- ☐ `getRole()` returns `"Student"` when called on a `Student` object, even via a `Person` reference
- ☐ `super.toString()` correctly delegates to the parent's version
- ☐ A `final` method in parent cannot be overridden (test and confirm the error)

Common Mistakes

Mistake	Clarification
Confusing overriding with overloading	Different parameter list = overloading, not overriding
Changing return type incompatibly	Return type must be the same or a covariant (subtype)
Overriding <code>static</code> methods	Static methods are hidden, not overridden — polymorphism does not apply

Extension Challenges

- **Basic:** `Shape` → `Circle` — override `toString()` to include the radius
- **Intermediate:** `Shape` → `Circle`, `Rectangle`, `Triangle` — each overrides `toString()` and a `getArea()` method
- **Advanced:** Create an array of `Shape[]` containing mixed types — loop and print each; explain why the correct `toString()` is called for each

Week 8 — 15 July – 21 July

★ MID-TERM EXAM ★

Exam Coverage (Weeks 1–7)

Topic	Key Areas
Introduction to OOP	OOP vs procedural, 4 pillars, real-world modeling
Classes & Objects	Class definition, object creation, independent state
Attributes & Methods	Instance vs local variables, return types, state changes
Encapsulation	Access modifiers, getters/setters, data validation
Constructors	Default, parameterized, overloading, <code>this()</code> chaining
Inheritance	IS-A relationship, <code>extends</code> , <code>super()</code> , <code>protected</code>
Overriding	<code>@Override</code> , <code>super.method()</code> , overriding vs overloading

Sample Exam Questions (Preview)

- [15 pts]** Define and explain Encapsulation. Write a `BankAccount` class with `private double balance`, a validated `deposit()` method, and a `getBalance()` getter.
- [20 pts]** Design a class hierarchy: `Vehicle → Car → ElectricCar`. Each class must add meaningful fields and constructors. Override `toString()` at each level using `super.toString()`.
- [15 pts]** Explain the difference between method overriding and method overloading. Provide one Java code example for each.
- [10 pts]** Identify and fix 5 errors in the provided code snippet.

Exam Logistics

- Duration: 1 hour
- Format: Closed book
- Total Points: 60

Week 9 — 22 July – 28 July

Polymorphism

Objectives

- Define polymorphism and explain its two forms
- Apply upcasting and understand its purpose
- Safely downcast using `instanceof`
- Write polymorphic code that works with parent references

Core Concept

Polymorphism (Greek: "many forms") means that the same operation can behave differently depending on the actual type of the object. A parent-type reference can point to a child-type object, and the correct overridden method is called at runtime.

Two Forms of Polymorphism

1. COMPILE-TIME (Static) Polymorphism
 - Method Overloading
 - Same method name, different parameter lists
 - Resolved when code is compiled
2. RUNTIME (Dynamic) Polymorphism
 - Method Overriding + Upcasting
 - Parent reference holds child object
 - Java decides which method to call at runtime

Upcasting & Downcasting

```
UPCASTING (safe, automatic)
Student s = new Student("Sokha", 20, "CS");
Person p = s;                                // OK – no cast needed
p.introduce();                                // calls Student's version if overridden

DOWNCASTING (requires explicit cast + instanceof check)
if (p instanceof Student) {
    Student s2 = (Student) p;                 // safe downcast
    s2.study();                               // child-only method now accessible
}
```

Java Illustration

```
public class Main {
    public static void main(String[] args) {

        // Polymorphic array – parent type holds child objects
        Person[] people = {
            new Student("Sokha", 20, "CS"),
            new Teacher("Syuth", 30, "OOP"),
            new Student("Dara", 22, "Engineering"),
        };

        // Java calls the correct overridden toString() at runtime
        for (Person p : people) {
            System.out.println(p.toString());
            System.out.println("Role: " + p.getRole());
        }

        System.out.println("---");

        // Safe downcasting
        for (Person p : people) {
            if (p instanceof Student s) { // Java 16+ pattern matching
                s.study();
            } else if (p instanceof Teacher t) {
                t.teach();
            }
        }
    }
}
```

Test Checkpoint

- ☐ A `Student` object assigned to a `Person` variable compiles without error
- ☐ The correct overridden method is called on the parent reference
- ☐ Calling a child-only method on a parent reference produces a compile error
- ☐ `ClassCastException` is avoided using `instanceof` before downcasting

Common Mistakes

Mistake	Clarification
Downcasting without <code>instanceof</code>	<code>ClassCastException</code> at runtime if the object is the wrong type
Calling child-only method on parent	

reference	Compile error — the parent type does not know about child-only methods
Confusing compile-time and runtime polymorphism	Overloading is resolved at compile time; overriding at runtime

Extension Challenges

- **Basic:** Array of `Animal[]` with `Cat`, `Dog`, `Bird` — call `makeSound()` polymorphically
- **Intermediate:** Write a `printAll(Person[] people)` static method that handles any mix of `Student` and `Teacher` objects
- **Advanced:** Explain in writing why polymorphism reduces the need to change calling code when new subtypes are added

Week 10 — 29 July – 4 August

Abstraction — Abstract Classes

Objectives

- Define abstraction as an OOP principle
- Distinguish abstract classes from concrete classes
- Write abstract methods and understand the enforcement contract
- Decide when to use an abstract class vs a concrete class

Core Concept

Abstraction means exposing only the relevant aspects of an object and hiding unnecessary internal details. An **abstract class** defines a common template with some behavior left intentionally unimplemented — forcing subclasses to fill in the details.

Real-World Analogy

"Shape" is an abstract concept.
 You cannot draw a "Shape" — only a Circle, Rectangle, or Triangle.
 But all shapes share common properties: they have a color, an area, and a perimeter.

Abstract Class Rules

abstract class

- Cannot be instantiated directly (new Shape() → Error)
- CAN have concrete methods (shared behavior)
- CAN have abstract methods (must be overridden by subclass)
- CAN have constructors (called via super() from subclass)
- Subclass MUST implement all abstract methods (unless the subclass is also abstract)

Java Illustration

```
public abstract class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    // Abstract methods - subclass MUST implement
    public abstract double getArea();
    public abstract double getPerimeter();

    // Concrete method - shared by all shapes
    public void printInfo() {
        System.out.printf("Shape: %-10s | Color: %-8s | Area: %.2f%n",
            getClass().getSimpleName(), color, getArea());
    }
}

public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    @Override public double getArea() { return Math.PI * radius * radius; }
    @Override public double getPerimeter() { return 2 * Math.PI * radius; }
}

public class Rectangle extends Shape {
    private double width, height;

    public Rectangle(String color, double width, double height) {
        super(color);
    }
}
```

```

        this.width = width; this.height = height;    }
    @Override public double getArea()        { return width * height; }
    @Override public double getPerimeter() { return 2 * (width + height); }}

```

Test Checkpoint

- ☐ `new Shape()` produces a compile error
- ☐ A subclass that does not implement all abstract methods also produces a compile error
- ☐ `printInfo()` (concrete) works on all subclass objects without modification
- ☐ Polymorphic array `Shape[]` calls the correct `getArea()` per type

Common Mistakes

Mistake	Clarification
Trying to instantiate an abstract class	Abstract classes exist only as base types
Abstract method with a body	Abstract methods have no body — the <code>{}</code> is illegal
Forgetting to implement all abstract methods	Partial implementation requires the subclass to also be declared <code>abstract</code>

Extension Challenges

- **Basic:** Add `Triangle extends Shape` — override `getArea()` (formula: $0.5 \times \text{base} \times \text{height}$)
- **Intermediate:** Design `abstract class Employee` with `abstract double calculateSalary()` — implement `FullTimeEmployee` and `PartTimeEmployee`
- **Advanced:** Write a loop that processes an `Employee[]` array and prints total payroll — discuss why this design is flexible when new employee types are added

Week 11 — 5 August – 11 August

Abstraction — Interfaces

Objectives

- Define an interface and explain its role as a contract
- Implement one or more interfaces in a class
- Distinguish between abstract class and interface

- Understand `default` methods (Java 8+)

Core Concept

An **interface** is a pure contract — it defines what a class must do, without specifying how. Unlike abstract classes, interfaces represent a **CAN-DO** relationship, not an IS-A relationship. A class can implement multiple interfaces.

Abstract Class vs Interface

	Abstract Class	Interface
Relationship	IS-A	CAN-DO
Multiple inherit?	No (single only)	Yes (many)
Constructor?	Yes	No
Instance fields?	Yes	No (constants only)
Concrete methods?	Yes	Yes (via default/static)
Main use	Shared base behavior	Capability contract

Java Illustration

```
// Interface - defines a contract, not an implementation
public interface Printable {
    void printDetails();
}

public interface Gradable {
    double calculateGrade();

    // Default method - optional to override (Java 8+)
    default String getLetterGrade() {
        double g = calculateGrade();
        if (g ≥ 90) return "A";
        if (g ≥ 80) return "B";
        if (g ≥ 70) return "C";
        return "F";
    }
}

// Student extends Person AND implements two interfaces
public class Student extends Person implements Printable, Gradable {
    private double[] scores;

    public Student(String name, int age, double[] scores) {
        super(name, age);
        this.scores = scores;
    }
}
```

```

@Override    public void printDetails() {
System.out.println("Student: " + name + " | Grade: " + getLetterGrade());    }
@Override    public double calculateGrade() {        double sum = 0;
for (double s : scores) sum += s;        return sum / scores.length;    }    }

```

Test Checkpoint

- ☐ Interface cannot be instantiated directly
- ☐ Class implements two interfaces simultaneously
- ☐ `default` method works without being overridden
- ☐ Interface reference variable: `Printable p = new Student( ... )` compiles and calls the correct method

Common Mistakes

Mistake	Clarification
Using <code>extends</code> instead of <code>implements</code>	Classes <code>implement</code> interfaces; they <code>extend</code> classes
Adding instance fields to an interface	Interface fields are implicitly <code>public static final</code> (constants)
Choosing interface when shared state is needed	Interfaces carry no state — use abstract class if shared fields are needed

Extension Challenges

- **Basic:** Create `Flyable` interface with `fly()` — implement in `Bird` and `Airplane`
- **Intermediate:** Create `Serializable` and `Loggable` interfaces — implement both in a `UserActivity` class
- **Advanced:** Refactor the `Shape` hierarchy: keep the abstract class for shared state (`color`), but extract `Drawable` and `Resizable` as interfaces. Discuss the design decision.

Week 12 — 12 August – 18 August

Object Relationships: Association, Aggregation & Composition

Objectives

- Identify and distinguish the three types of OOP relationships

- Model HAS-A relationships correctly
- Understand object lifecycles in aggregation vs composition
- Implement relationships using object references

Core Concept

Not all OOP relationships are inheritance (IS-A). Objects also relate to each other through **HAS-A** connections. The strength of that connection determines whether it is Association, Aggregation, or Composition.

Relationship Types

ASSOCIATION (uses)

Class A uses Class B, but neither owns the other.

Example: Student — Course (a student is enrolled in a course; either can exist independently)

AGGREGATION (has-a, weak ownership)

Class A has Class B, but B can exist without A.

Example: Department —◇ Teacher (a teacher can exist without a department)

Symbol: open diamond ◇

COMPOSITION (has-a, strong ownership)

Class A owns Class B – B cannot exist without A.

Example: House —◆ Room (rooms don't exist independently of the house)

Symbol: filled diamond ◆

Java Illustration

```
// Composition – Room cannot exist without a House
public class Room {
    private String type;    // e.g., "Bedroom", "Kitchen"
    private double size;

    public Room(String type, double size) {
        this.type = type;
        this.size = size;
    }
    public String toString() { return type + " (" + size + " m²)"; }
}

public class House {
    private String address;
    // Rooms are created inside House – they live and die with it (composition)
```

```

public House(String address) {      this.address = address;    }
public void addRoom(String type, double size) {
    rooms.add(new Room(type, size)); // House owns Room creation }
public void printLayout() {        System.out.println("House at " + address);
    for (Room r : rooms) System.out.println("  - " + r);    } }
// Aggregation – Teacher can exist without a Department
public class Teacher {
    private String name;    public Teacher(String name) { this.name = name; }
    public String getName()    { return name; }}                public class Department {
    private String name;    private List<Teacher> teachers = new ArrayList<>(); // aggregated
        public Department(String name) { this.name = name; }
    public void addTeacher(Teacher t) {
        teachers.add(t); // Teacher was created externally – Department just references it
    } }

```

Test Checkpoint

- ☐ Can correctly classify a given relationship as association, aggregation, or composition
- ☐ Composition: child objects are created inside the parent — not passed in
- ☐ Aggregation: child objects are passed into the parent from outside
- ☐ Can draw a simple UML-style diagram showing the relationship type

Common Mistakes

Mistake	Clarification
Using inheritance for HAS-A	<code>Engine extends Car</code> is wrong — Car HAS-A engine, not IS-A engine
Treating all HAS-A as composition	Ask: can the child exist without the parent? If yes → aggregation
Circular references	$A \rightarrow B \rightarrow A$ creates memory and design problems

Extension Challenges

- **Basic:** Model a `Library` that has `Book` objects (composition) — library creates and owns books
- **Intermediate:** Model a `University` that has `Department` objects, each with a list of `Teacher` objects (aggregation) — teachers can be shared across departments
- **Advanced:** Draw a complete relationship diagram for a university system (`University`, `Department`, `Course`, `Teacher`, `Student`, `Enrollment`) — label each relationship type

Exception Handling in OOP

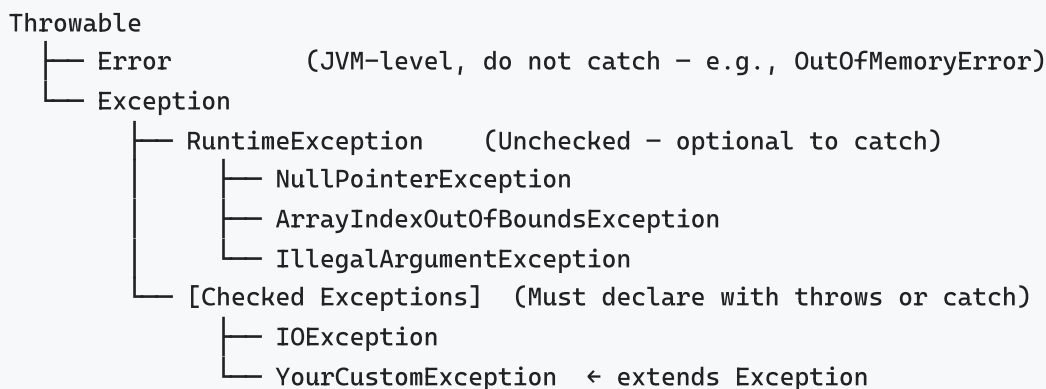
Objectives

- Understand exceptions as OOP objects (part of a class hierarchy)
- Use `try`, `catch`, `finally` blocks correctly
- Create custom exception classes
- Apply `throws` declarations in method signatures

Core Concept

In OOP, **exceptions are objects** — instances of classes that extend `Throwable`. Exception handling is a structured way to respond to errors without breaking the object model. Custom exceptions allow you to express domain-specific errors clearly.

Exception Class Hierarchy



Java Illustration

```
// Custom domain exception - an OOP class for a specific error
public class InsufficientFundsException extends Exception {
    private final double shortfall;

    public InsufficientFundsException(double shortfall) {
        super("Insufficient funds. Short by: $" + shortfall);
        this.shortfall = shortfall;
    }
}
```



```

        public double getShortfall() { return shortfall; }
    }

    public class BankAccount {
        private String owner;
        private double balance;

        public BankAccount(String owner, double initialBalance) {
            this.owner = owner;
            this.balance = initialBalance;
        }

        public void withdraw(double amount) throws InsufficientFundsException {
            if (amount > balance) {
                throw new InsufficientFundsException(amount - balance);
            }
            balance -= amount;
            System.out.println("Withdrawn: $" + amount + " | New balance: $" + balance);
        }
    }

    public class Main {
        public static void main(String[] args) {
            BankAccount acc = new BankAccount("Sokha", 500.0);
            try {
                acc.withdraw(200.0);
                acc.withdraw(400.0); // triggers exception
            } catch (InsufficientFundsException e) {
                System.out.println("Error: " + e.getMessage());
                System.out.println("Shortfall: $" + e.getShortfall());
            } finally {
                System.out.println("Transaction process finished.");
            }
        }
    }

```

Test Checkpoint

- ☐ `finally` block executes regardless of whether an exception was thrown
- ☐ Custom exception message appears correctly in `e.getMessage()`
- ☐ Method with `throws` declaration compiles only when caller handles or re-throws
- ☐ Unchecked exception (`IllegalArgumentException`) does not require `throws` declaration

Common Mistakes

Mistake	Clarification
Catching <code>Exception</code> for everything	Too broad — hides bugs; catch specific types
Empty <code>catch</code> block	Silently swallows errors — always log or rethrow
Using exceptions for normal flow control	Exceptions are for exceptional situations, not <code>if</code> replacement

Extension Challenges

- **Basic:** Write a method `parseInt(String s)` that throws `IllegalArgumentException` if the input is not numeric
- **Intermediate:** Create `StudentNotFoundException` and use it in a `findStudentById()` method
- **Advanced:** Design a layered exception system: `RepositoryException` (base) → `StudentNotFoundException`, `CourseFullException` — propagate up through a service layer

OOP Design Principles (SOLID Introduction)

Objectives

- Explain the five SOLID principles at a conceptual level
- Identify violations of SOLID in provided code examples
- Apply Single Responsibility and Open/Closed principles in practice
- Understand how SOLID relates to the four OOP pillars

Core Concept

SOLID is a set of five design principles that lead to more maintainable, flexible, and scalable OOP systems. They do not replace the four pillars — they guide how to apply them well.

SOLID Overview

S – Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change.

Example: Do not put PDF generation AND database saving in the same class.

O – Open/Closed Principle (OCP)

A class should be open for extension, closed for modification.

Example: Add new payment types by creating new subclasses – not by editing the existing class.

L – Liskov Substitution Principle (LSP)

A subclass should be substitutable for its parent without breaking behavior.

Example: If Person works in a system, Student (a Person) must work correctly too.

I – Interface Segregation Principle (ISP)

Do not force a class to implement interfaces it does not use.

Example: Split a large Workable interface into Printable, Savable, Exportable.

D – Dependency Inversion Principle (DIP)

Depend on abstractions, not on concrete implementations.

Example: A service class depends on a Repository interface, not a specific MySQLRepository.

SRP Violation vs Correct Design

```
// VIOLATION - one class does too many things
public class Student {
    public void study() { ... }
    public void saveToDatabase() { ... } // database concern
    public void exportToPDF() { ... }   // export concern
    public void sendEmail() { ... }     // notification concern
}

// CORRECT - each class has one responsibility
public class Student { public void study() { ... } }
public class StudentRepository { public void save(Student s) { ... } }
public class StudentExporter { public void exportToPDF(Student s) { ... } }
public class NotificationService { public void sendEmail(Student s) { ... } }
```

Test Checkpoint

- ☐ Can identify the SOLID principle violated in a given class
- ☐ Can refactor a class that has multiple responsibilities into two or more classes
- ☐ Can explain LSP in plain English with an example
- ☐ Can design a system where adding a new type does not require modifying existing classes (OCP)

Common Mistakes

Mistake	Clarification
Treating SOLID as rules to follow blindly	They are principles — apply with judgment based on context
Over-engineering small projects	SOLID pays off at scale; don't over-apply for a 50-line program
Confusing ISP and SRP	SRP = one reason to change per class; ISP = fine-grained interfaces

Extension Challenges

- **Basic:** Find 2 SOLID violations in a provided code snippet — name the principle and explain why
- **Intermediate:** Refactor a `ReportManager` class (saves, exports, emails, displays) into SOLID-compliant classes
- **Advanced:** Design a payment processing system using OCP — new payment types (Crypto, QR) should be addable without modifying the `PaymentProcessor` class

Introduction to Design Patterns & Course Review

Objectives

- Understand what design patterns are and why they matter
- Apply Singleton and Builder patterns in Java
- Connect design patterns back to the four OOP pillars
- Complete a mini review of all 14 weeks

Core Concept

Design Patterns are proven, reusable solutions to commonly occurring OOP design problems. They are not copy-paste code — they are conceptual templates. Knowing them helps you communicate design decisions and avoid reinventing the wheel.

Three Pattern Categories

CREATIONAL	STRUCTURAL	BEHAVIORAL
How objects are created	How objects are composed	How objects communicate
Singleton	Adapter	Observer
Builder	Decorator	Strategy
Factory	Facade	Command

Singleton Pattern

```
// Ensures only ONE instance of a class exists globally
public class AppConfig {
    private static AppConfig instance;
    private String dbUrl;

    private AppConfig() {
        this.dbUrl = "jdbc:mysql://localhost/university_db";
    }

    public static AppConfig getInstance() {
        if (instance == null) {
            instance = new AppConfig();
        }
    }
}
```

```

    }        return instance;    }
    public String getDbUrl() { return dbUrl; }}
// Usage - always the same object AppConfig config1 = AppConfig.getInstance();
AppConfig config2 = AppConfig.getInstance(); System.out.println(config1 == config2); // true

```

Builder Pattern

```

// Constructs complex objects step by step
public class StudentBuilder {
    private String name;
    private int    age    = 18;
    private String major = "Undeclared";
    private double gpa    = 0.0;

    public StudentBuilder name(String v) { this.name = v; return this; }
    public StudentBuilder age(int v)     { this.age  = v; return this; }
    public StudentBuilder major(String v) { this.major = v; return this; }
    public StudentBuilder gpa(double v)  { this.gpa  = v; return this; }
    public Student build()                { return new Student(name, age, major, gpa); }
}

// Clean, readable object creation
Student s = new StudentBuilder()
    .name("Sokha")
    .age(21)
    .major("Computer Science")
    .gpa(3.8)
    .build();

```

Course Review Checklist (All Weeks)

Pillar / Topic	Key Concept	Week
OOP Overview	4 pillars, OOP vs Procedural	1
Classes & Objects	Blueprint vs instance, memory model	2
Attributes & Methods	State, behavior, return types	3
Encapsulation	<code>private</code> fields, getters/setters, validation	4
Constructors	Initialization, overloading, <code>this()</code>	5
Inheritance	IS-A, <code>extends</code> , <code>super()</code> , <code>protected</code>	6

Method Overriding	<code>@Override</code> , <code>super.method()</code> , <code>final</code>	7
Polymorphism	Upcasting, downcasting, runtime dispatch	9
Abstraction	Abstract classes, interfaces, contracts	10–11
Relationships	Association, Aggregation, Composition	12
Exception Handling	Custom exceptions, try/catch/finally	13
SOLID Principles	SRP, OCP, LSP, ISP, DIP	14
Design Patterns	Singleton, Builder, Factory (intro)	15

Week 16 — 9 September – 15 September

★ FINAL EXAM ★

Exam Coverage (Weeks 1–15)

Topic Area	Weight
OOP Foundations (classes, objects, state, constructors)	20%
Encapsulation & Inheritance	20%
Polymorphism	15%
Abstraction (abstract classes + interfaces)	20%
Object Relationships	10%
Exception Handling & SOLID Principles	10%
Design Patterns (Singleton, Builder)	5%

Sample Final Questions (Preview)

- [25 pts]** Design a complete `UniversitySystem` using all four OOP pillars:
 - `abstract Person` (Encapsulation + Abstraction)
 - `Student` and `Teacher` extend `Person` (Inheritance)

- Polymorphic method `printAll(Person[] people)` (Polymorphism)
- 2. **[20 pts]** Classify the relationships in a given class diagram as Association, Aggregation, or Composition — justify each answer.
- 3. **[20 pts]** Identify all SOLID violations in a provided `OrderManager` class. Refactor it into a correct design.
- 4. **[15 pts]** Write a `StudentRepository` class using the Singleton pattern — it holds a list of `Student` objects with `add()`, `findById()`, and `getAll()` methods, with custom exception `StudentNotFoundException`.
- 5. **[20 pts]** Multiple choice and short-answer questions covering OOP concepts, terminology, and diagrams.

Final Exam Logistics

- Duration: 1 hour
- Format: Closed book
- Total Points: 100

Recommended Reading

Resource	Type	Access
<i>Introduction to Programming Using Java</i> — David J. Eck	Free PDF / HTML	https://math.hws.edu/javanotes/
<i>Object Oriented Programming Using Java</i> — Simon Kendal	Free PDF	https://www.minds.co.za/wp-content/uploads/2019/06/object-oriented-programming-using-java.pdf
<i>Java, Java, Java</i> — Morelli & Walde	Open Textbook	https://open.umn.edu/opentextbooks/textbooks/java-java-java-object-oriented-problem-solving
<i>Fundamentals of OOP and Data Structures in Java</i> —		

Wiener & Pinson	PDF	https://library.uc.edu.kh/userfiles/pdf/8.Fundamentals%20of%20OOP%20and%20Data%20Structure
Oracle Java OOP Tutorial	Official Reference	https://docs.oracle.com/javase/tutorial/java/concepts/

Course Outline — Object-Oriented Programming Concepts
 IIC University of Technology | @KhmerSide | syuthd.com | 2026